

Edge Agent Runtime Design

Backend Intern Assignment — Sarvam AI

Submitted by

Ansh Singhal

anshsinghal3107@gmail.com · +91 89295 54991

Date: 19 May 2026

GitHub Repository: github.com/AnshSinghal/edge-ai

All detailed documentation, architecture diagrams, SVG assets, and Postman API collections are available in the repository linked above.

Table of Contents

Part A — Core Architecture

1. [Part A §1 — Process Architecture](#)
 - Process Model (4 processes, supervisor tree)
 - Worker Crash Recovery (0–500ms sequence)
 - References & Postman Collection
2. [Part A §2 — Worker Crash Recovery](#)
 - Recovery sequence: detection → decision → respawn → warm reload
 - Warm vs cold restart path
 - In-flight request handling
3. [Part A §3 — Qualcomm NPU Failure Handling](#)
 - ERROR_DEVICE_REMOVED detection
 - CPU-only fallback invocation
 - Latency degradation signalling
 - Quarantine and health probe
4. [Part A §4 — Apple Neural Engine Fallback](#)
 - 4-tier escalation: ANE → Metal GPU → CPU → Cloud
 - Selection logic and decision tree
 - Failure escalation order
 - API contract preservation

Part B — Micro-Frontend Integration

5. [Part B §1 — Agent Integration Layer](#)
 - Shell-owned singleton design
 - Tradeoffs vs direct MFE access
 - Security and resource coordination
6. [Part B §2 — Client-Side Request Scheduler](#)
 - WFQ priority queue (P0/P1/P2, weights 4:2:1)
 - Per-MFE fairness and quota
 - Cancellation, timeout, backpressure
7. [Part B §3 — Streaming + Capacity Contention](#)
 - 4/4 slot scenario: Meeting Summariser + Doc Q&A
 - UX contract: events, states, queue visibility
 - Cancellation, retry, recovery sequences

Supplemental

- [API Reference & Postman Collections](#)
- [Design Assumptions](#)

Part A §1 — Process Architecture

Full document: [PART-A-PROCESS-ARCHITECTURE.md](#) Postman collection: [edge-agent-api.postman_collection.json](#)

1.1 Process Model

The Edge Agent spawns **4 processes** on startup using a **supervisor-tree** pattern. Each runs in a separate OS address space; a crash in any child is contained without affecting siblings.

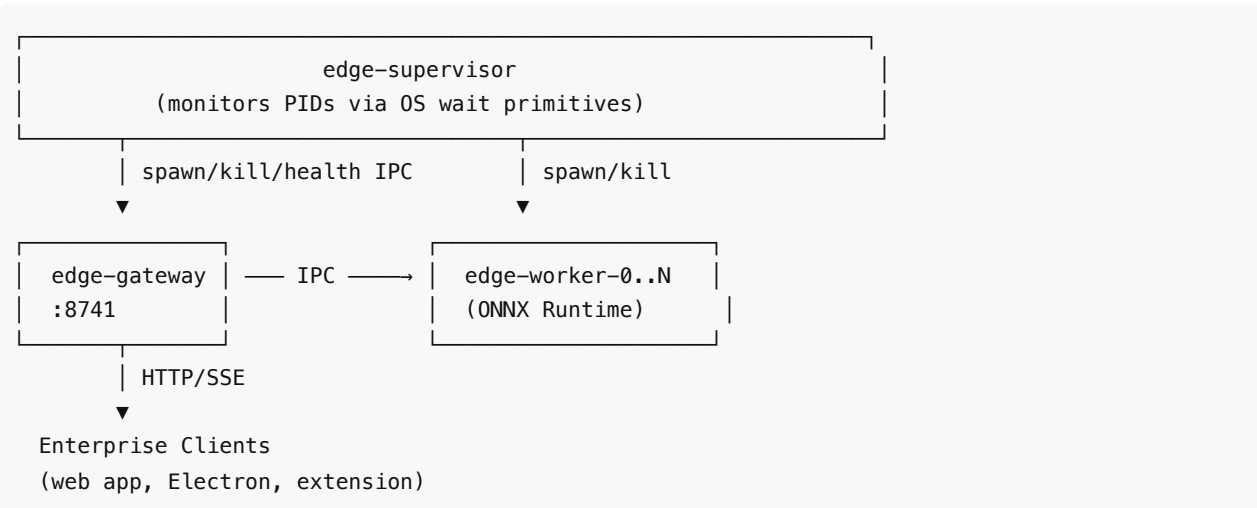
Process	Type	Responsibility
edge-supervisor	Persistent	Process lifecycle, crash recovery, health orchestration
edge-gateway	Persistent	HTTP/WebSocket serving, auth, routing, response streaming
edge-model-mgr	On-demand	Model download, integrity verification, disk cache
edge-worker-N	Pooled (0-4)	AI model execution (ONNX Runtime), one model per worker

Core constraint: The inference worker and API-serving process must be isolated so a crash in one cannot bring down the other. They communicate via OS-native IPC (Named Pipes on Windows, Unix Domain Sockets on macOS) — not shared memory, not TCP.

Key design decisions:

- edge-supervisor is ~200 lines with no business logic — minimal crash surface
- edge-gateway is stateless — a crash causes ~200ms downtime, no inference state lost
- edge-worker-N is pooled 0-4 — model residency in OS page cache enables warm restarts
- Restart budget:** max 5 restarts per 60s per child. Budget exceeded → QUARANTINED state, health alert emitted

1.2 Communication Architecture



Architecture diagram: See `process-architecture.svg` in the repository.

Part A §2 — Worker Crash Recovery

Full document: [PART-A2-WORKER-CRASH-RECOVERY.md](#) Postman collection: [edge-agent-a2-crash-recovery.postman_collection.json](#)

Scenario: `edge-worker-0` crashes mid-request with Windows access violation `0xC0000005` .

2.1 Recovery Sequence (0 → 500ms)

Time	Event	Component	Action
t = 0ms	0xC0000005 terminates worker	OS	Two simultaneous events
t = 1–5ms	WaitForSingleObject / waitpid unblocks	Supervisor	Reads exit code, starts budget check
t = 1–3ms	WriteFile returns ERROR_BROKEN_PIPE	Gateway	Marks worker-0 dead in routing table
t = 5–15ms	Budget check passes (<5 crashes/60s)	Supervisor	Clears for respawn
t = 5–15ms	In-flight request routing decision	Gateway	Re-dispatch or 503
t = 15–50ms	CreateProcess / posix_spawn	Supervisor	New edge-worker-0 spawns
t = 50–500ms	Warm model reload via mmap	Worker-0	Page cache hit; no disk I/O
t = 500ms	Worker-0 IDLE, re-enters pool	Gateway	Normal routing resumes

2.2 In-Flight Request Handling

The gateway makes a routing decision at t = 5–15ms :

Pool state	Gateway action	Client experience
Another worker idle, same model	Transparent re-dispatch	~50ms extra latency
Another worker, different model	Re-dispatch with model swap	2–5s extra (model load)
No workers available	503 Service Unavailable + Retry-After: 1	Error, must retry

2.3 Warm Restart vs Cold Start

Path	Trigger	Duration	Mechanism
Warm restart	Normal crash, model in page cache	50–500ms	mmap — kernel re-maps pages already in RAM
Cold start	First boot, evicted cache, new model	2–5s	Full disk read + NPU/GPU load

The OS page cache retains model data after a process exits. A respawned worker re-maps the same pages — no disk I/O on a warm restart. This is the primary performance optimisation.

State persisted to disk: `crash_log.jsonl` (crash metadata), restart budget counter (survives supervisor restarts). Inference is stateless — no partial results or checkpoints.

Recovery diagram: See `worker-crash-recovery.svg` in the repository.

Part A §3 — Qualcomm NPU Failure Handling

Full document: [PART-A3-NPU-FALLBACK.md](#) Postman collection: [edge-agent-a3-npu-fallback.postman_collection.json](#)

Scenario: Snapdragon X Elite system. NPU driver returns `DXGI_ERROR_DEVICE_REMOVED` during inference.

Key principle: This is **not** a process crash. The worker process stays alive. Recovery is entirely in-process.

3.1 Recovery Timeline

Time	Event	Client impact
t = 0ms	DXGI_ERROR_DEVICE_REMOVED returned	Request in progress
t = 1ms	Device Manager marks NPU FAULTED	None
t = 2ms	NPU ONNX session destroyed	None
t = 5–20ms	CPU ONNX session created (model already mmap'd)	None
t = 20ms	Same request retried on CPU EP	None — client still waiting
t = 70–270ms	CPU inference completes (~3× slower)	Response with X-Edge-Compute: cpu-fallback
t = 270ms+	Worker heartbeat: device: cpu, throughput adjusted	Subsequent requests ~3× slower
t = 30s	First health probe: test tensor on fresh NPU session	Background
t = 30s+	Probe passes → NPU restored. Probe fails → backoff 60s → 120s	Normal or extended fallback

Total impact on the failing request: ~200ms extra latency. Client receives a valid result, not an error.

3.2 Failure Detection

Detection is synchronous — same thread, same request lifecycle:

```
auto status = session.Run(run_options, input_names, inputs, output_names, outputs);
if (status.Code() == ORT_FAIL) {
    HRESULT hr = dxgi_device->GetDeviceRemovedReason();
    if (hr == DXGI_ERROR_DEVICE_REMOVED || hr == DXGI_ERROR_DEVICE_RESET) {
        device_manager.quarantine(DeviceType::NPU, hr);
    }
}
```

3.3 Client Communication

The API contract does not change. Clients detect fallback via response headers:

Header	Normal	NPU fallback
X-Edge-Compute	npu	cpu-fallback
X-Edge-Latency-Ms	~90	~270
HTTP status	200	200

3.4 Quarantine and Health Probe

- NPU enters `FAULTED` state on `DEVICE_REMOVED` — no retries until probe passes
- Health probe: fresh ONNX session + small test tensor, background thread
- Backoff: 30s → 60s → 120s → 240s (cap)
- Quarantine is **in-memory only** — intentional. Re-detection after a worker restart costs one failed request (~200ms), acceptable vs disk persistence complexity

Fallback diagram: See `npu-fallback-sequence.svg` in the repository.

Part A §4 — Apple Neural Engine Fallback

Full document: [PART-A4-ANE-FALLBACK.md](#) Postman collection: [edge-agent-a4-ane-fallback.postman_collection.json](#)

Scenario: Core ML returns `kCMErrorUnsupportedOperation` — ANE claimed by another process, thermally throttled, or model op is ANE-incompatible.

Key difference from NPU scenario: ANE unavailability is often **transient**. The fallback uses tiered escalation with independent health tracking per tier.

4.1 Fallback Tier List

Priority	Tier	Provider	Core ML Units	When chosen
0	ANE	Apple Neural Engine	.all	Default
1	Metal GPU	Apple GPU via Metal	.cpuAndGPU	ANE faulted, GPU ready, utilisation < 80%
2	CPU	Accelerate (AMX + NEON)	.cpuOnly	ANE + GPU both faulted, or non-latency-critical
3	Cloud	Sarvam cloud API	N/A (HTTP)	Latency-critical + local too slow + enterprise policy

4.2 Recovery Timeline

Time	Trigger	Tier	Latency factor
t = 0ms	kCMErrorUnsupportedOperation	0 → 1 (Metal GPU)	1.8×
t = 50ms	GPU: Metal shader compilation fails (rare)	1 → 2 (CPU)	4.4×
t = 250ms	CPU succeeds — always stable	2 (stable)	4.4×
t = 30s+	Probe: GPU restored	2 → 1	1.8×
t = 120s+	Probe: ANE restored	1 → 0	1.0×

4.3 Client Communication

API contract is preserved. Fallback visible only via headers:

Header	ANE	Metal GPU	CPU	Cloud
X-Edge-Compute	ane	metal-gpu	cpu	cloud
X-Edge-Tier	0	1	2	3
HTTP status	200	200	200	200

Cloud fallback additionally sets `X-Edge-Privacy: off-device` — MFEs can use this to display a consent notice.

4.4 Performance Tradeoffs

Tier	Latency vs ANE	Power	Notes
------	----------------	-------	-------

ANE	1.0× (baseline)	Lowest	Dedicated silicon, shared system resource
Metal GPU	1.8×	Medium	Apple Silicon integrated GPU, excellent for matrix ops
CPU	4.4×	Higher	AMX2 + NEON; always available, cannot fail
Cloud	8–30× (network)	Lowest (local)	Requires connectivity, exits device security boundary

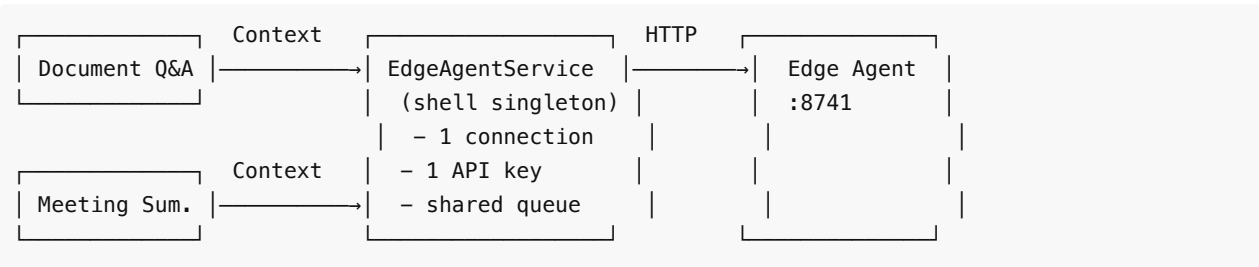
Fallback decision tree: See `ane-fallback-tree.svg` in the repository.

Part B §1 — Agent Integration Layer

Full document: [PART-B-MFE-INTEGRATION.md](#) Postman collection: [edge-agent-b-mfe-integration.postman_collection.json](#)

1.1 The Decision: Shell-Owned Singleton

Answer: The shell owns the connection. MFEs never call `localhost:8741` directly.



1.2 Tradeoff Analysis

Concern	Direct MFE Access	Shell-Owned Singleton
API key exposure	Each MFE holds the key — supply chain attack leaks it	Key in shell only — MFEs never see it
Connection count	N MFEs × M WebSockets	Always 1 connection
Slot coordination	Race conditions on 4-slot limit	Scheduler enforces limit centrally
Failure handling	Each MFE reconnects independently, causes thundering herd	Shell reconnects once, MFEs unaffected
Cancellation	No cross-MFE coordination	Shell cancels on MFE unmount
Team coupling	Each team must implement auth + connection + error handling	Teams call <code>useEdgeAgent()</code> — one API

Security implication: Direct access exposes the API key to every loaded MFE module. Module Federation loads third-party chunks at runtime — a compromised MFE has full key access. Shell-owned means the key never leaves the shell's closure.

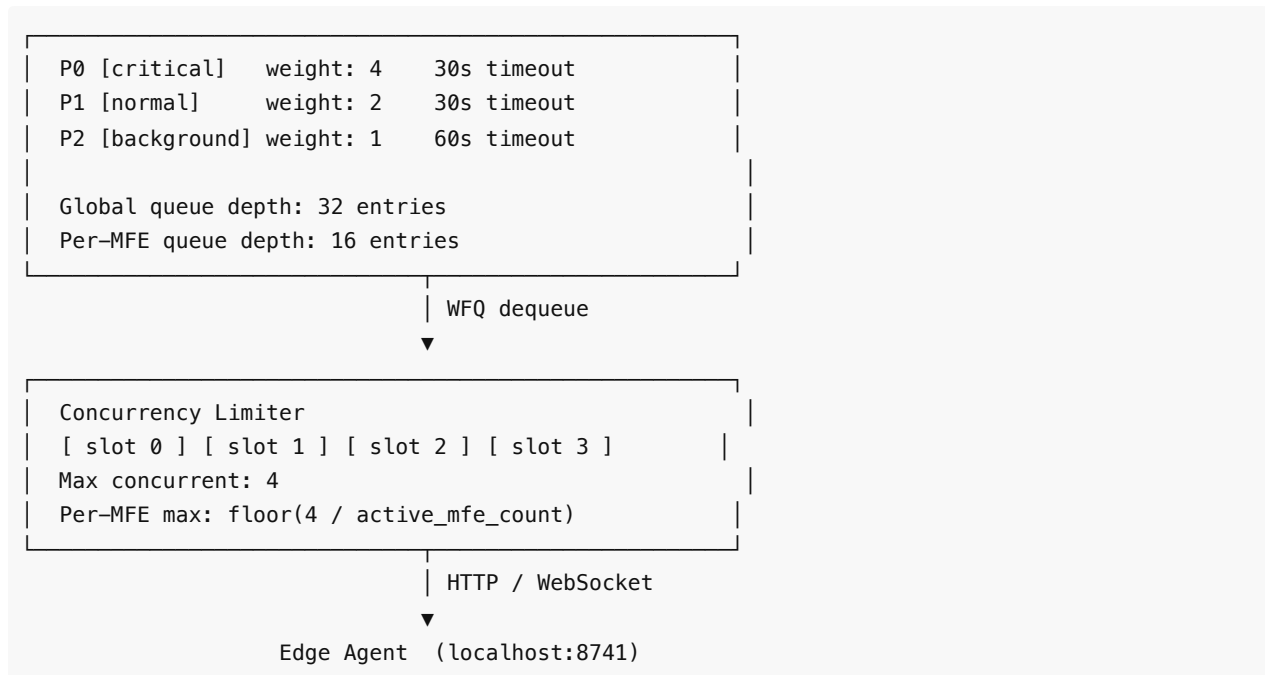
Resource coordination: The Edge Agent's 4-slot bounded queue returns `429` on overflow. Without a shared scheduler, MFEs race to fill slots and cause thundering-herd retries. The shell singleton is the only location that can enforce per-MFE fairness.

Part B §2 — Client-Side Request Scheduler

Full document: [PART-B2-CLIENT-SCHEDULER.md](#) Postman collection: [edge-agent-b2-scheduler.postman_collection.json](#)

2.1 Queue Structure

Three priority lanes with **Weighted Fair Queueing (WFQ)** — not strict priority — to prevent starvation:



2.2 Scheduling Policy

Weighted Fair Queueing: Slots are distributed across lanes by weight (4:2:1). P0 gets 4 of every 7 dispatch opportunities; P2 always gets at least 1. No lane can starve.

Per-MFE fairness: Each MFE gets $\text{floor}(4 / N)$ slots where N = number of active MFEs. With 2 MFEs: 2 slots each. One MFE cannot monopolise all 4 slots.

Age-based promotion: P1 requests waiting > 20s are promoted to P0. P2 requests waiting > 40s are promoted to P1. Prevents indefinite starvation even under sustained P0 load.

2.3 Concurrency Management

AIMD backpressure: If the Edge Agent returns 429, the scheduler halves its target concurrency (multiplicative decrease). On each successful 200, it increments by 1 (additive increase). This converges to the true agent capacity without hard-coded limits.

Scheduler states (6): IDLE → DISPATCHING → SATURATED → BACKPRESSURE → SUSPENDED → DRAINING

2.4 Cancellation and Timeout

Cancellation chain: MFE's AbortSignal → Scheduler AbortController → fetch AbortController → TCP RST to Edge Agent. TCP RST is required — a closed socket frees the agent's internal slot immediately. HTTP close without RST can leave the slot occupied until the agent's own timeout.

Timeout: Timer starts at inference() call (not at dispatch). A request that waits 25s in queue and takes 5s of inference has used its full 30s budget. Users don't distinguish queue time from inference time — a single timer matches UX expectations.

Part B §3 — Streaming + Capacity Contention

Full document: [PART-B3-STREAMING-CONTENTION.md](#) Postman collection: [edge-agent-b3-streaming-contention.postman_collection.json](#)

Scenario: Meeting Summariser holds 2 P0 SSE slots (ASR + TTS). 2 other slots occupied by batch. 4/4 full. User opens Document Q&A and submits a query.

3.1 Slot State at Contention

```
Edge Agent Slots (4/4 occupied):
  slot 0: meeting-sum  ASR stream (P0)  — SSE tokens —>
  slot 1: meeting-sum  TTS stream (P0)  — SSE audio  —>
  slot 2: [batch]       completing...    — HTTP 200   —>
  slot 3: [batch]       completing...    — HTTP 200   —>

Scheduler Queue:
  P0 [critical]  □ □ □ □  (empty)
  P1 [normal]   ■ □ □ □  (doc-qa query – just arrived)
  P2 [background] □ □ □ □  (empty)

Per-MFE counters:
  meeting-sum:  2/2 in-flight (at quota)
  doc-qa:        0/2 in-flight (waiting)
```

Doc Q&A is queued, not rejected: 4/4 is the expected steady state under load. Scheduler enqueues P1 and starts a 30s timeout.

Meeting Summariser is not interrupted: P1 cannot preempt P0 — cutting live transcription is irreversible damage.

3.2 UX Contract — What Each MFE Receives

Meeting Summariser — zero impact:

Property	Value
Token latency	Unchanged (~5–15ms per token)
Token ordering	Preserved (SSE is ordered, scheduler does not reorder)
Notification of contention	None — Meeting Summariser has no visibility into Doc Q&A

Document Q&A — queued state:

```
// Event received immediately on submission
{ event: 'scheduler:queued', request_id: 'req_001', position: 1, estimated_wait_ms: 500 }

// Event received when slot freed
{ event: 'scheduler:dispatched', request_id: 'req_001', queued_ms: 200, slot: 2 }
```

3.3 7 MFE-Observable Request States

State	Entry	MFE observes
SUBMITTED	MFE calls inference() / stream()	Promise pending
QUEUED	No slot available	scheduler:queued event with position
DISPATCHED	Slot freed, HTTP sent to agent	scheduler:dispatched event

STREAMING	SSE opens, first token arrives	Tokens via AsyncGenerator
COMPLETE	[DONE] received	Promise resolves { status: 'ok' }
CANCELLED	MFE aborts or unmounts	Promise abandoned
TIMED_OUT	30s elapsed since SUBMITTED	Promise resolves { status: 'error', code: 'timeout' }

3.4 Queue Visibility Rules

Observer	Sees	Cannot see
Meeting Summariser	Its own stream states only	Doc Q&A queue position
Document Q&A	Its own queue position and request states	Meeting Summariser slot count
Shell / DevTools	Full scheduler state (all MFEs)	N/A

Cross-MFE visibility would create coupling between teams, leak usage patterns, and add complexity with no UX benefit.

3.5 Cancellation Behaviour

Scenario A — Cancel while queued (no slot freed): Scheduler removes the P1 queue entry. No slot consumed — nothing released. `scheduler:cancelled { was_queued: true }`. Meeting Summariser: unaffected.

Scenario B — Cancel in-flight (slot freed): AbortController fires → TCP RST → agent drops request → slot freed → dispatch loop fires within 0.15ms. Meeting Summariser: unaffected.

Scenario C — Meeting Summariser stops (user stops recording): Both SSE streams aborted → 2 slots freed → quota recalculates (1 active MFE, doc-qa max = 4) → Doc Q&A dequeued and dispatched.

3.6 Retry Semantics

The scheduler never auto-retries. Retry decisions belong to the MFE — only the MFE knows if the user is still waiting and if a retry makes UX sense.

Error code	Retry recommended	MFE action
timeout	Yes (user-triggered)	Show "Timed out — Retry?" button
queue_full	Yes (after short wait)	Show "System busy — try again"
disconnected	Auto on reconnect	Subscribe to <code>connection:restored</code> , then re-submit
cancelled	No	User initiated — no action needed
preempted	Automatic	Scheduler re-queues at P2 lane head — invisible to MFE

API Reference & Postman Collections

All API examples and Postman collections are in the GitHub repository.

Collection	Covers	Requests
edge-agent-api	Core API (auth, inference, health, status)	18
edge-agent-a2-crash-recovery	Worker crash and recovery flows	20
edge-agent-a3-npu-fallback	NPU failure and CPU fallback	22

edge-agent-a4-ane-fallback	ANE 4-tier escalation	24
edge-agent-b-mfe-integration	MFE integration and EdgeAgentService	18
edge-agent-b2-scheduler	Scheduler: WFQ, fairness, backpressure	24
edge-agent-b3-streaming-contention	4/4 contention, SSE, cancellation	18

Base URL: `http://127.0.0.1:8741`

Auth: Authorization: Bearer <api_key> (key provisioned from OS keychain)

Design Assumptions

The following assumptions are stated explicitly throughout the detailed documents:

1. **Inference is stateless** — no partial results or checkpoints. Crashed requests must be re-submitted by the client.
2. **OS page cache persists model data** after worker process exit, enabling warm restarts without disk I/O.
3. **Client request_id is idempotent** — safe to re-dispatch the same request to a different worker.
4. **ONNX Runtime supports runtime EP swapping** (NPU → CPU) without process restart.
5. **ANE unavailability is often transient** — quarantine + probe recovery is the right strategy (unlike NPU `DEVICE_REMOVED` which is unrecoverable for the session).
6. **Per-MFE quota ($\text{floor}(4/N)$) prevents monopolisation** but does not preempt in-flight requests.
7. **Cloud fallback (Tier 3) is opt-in** via enterprise policy. It is disabled by default and requires explicit administrator consent.
8. **MFEs are untrusted** — developed by different teams, potentially compromised via supply chain. The shell treats them as consumers, not peers, and never exposes the API key.
9. **Single-tab queue** — `EdgeAgentService` is per-tab. Multiple tabs each get independent scheduler instances.
10. **estimated_wait_ms is best-effort** — calculated from a 20-response rolling average with a conservative (over-estimate) bias.

GitHub: github.com/AnshSinghal/edge-ai

Ansh Singhal · anshsinghal3107@gmail.com · +91 89295 54991